JSPM University Pune
Faculty of Science and Technology
School of Computational Sciences
M.Sc. Sem-2
Data Science and Artificial Intelligence(DSAI) /Cyber Security (C.S.)

EXPERIMENT NO: 2 **DATE: / /**

AIM : ARDUINO IDE SOFTWARE INSTALLATION AND PROGRAMMING CONCEPTS FOR ESP32

OBJECTIVE: The main objective of this experiment is,

A. Install Arduino.
B. Study Programming concept.

DESCRIPTION:

INSTALL ARDUINO IDE

The Arduino IDE (Integrated Development Environment) in reference to the ESP32 is a software platform that allows users to write, compile, and upload code (sketches) to ESP32 development boards. While the ESP32 is a powerful microcontroller with its own official Espressif SDK (ESP-IDF), the Arduino IDE offers a familiar and user-friendly environment for programming the ESP32, especially for those accustomed to the Arduino ecosystem.

Arduino IDE installation steps:

STEP 1: Download Arduino IDE: Navigate to the official Arduino website and download the appropriate version of the Arduino IDE for your operating system (Windows, macOS, Linux).

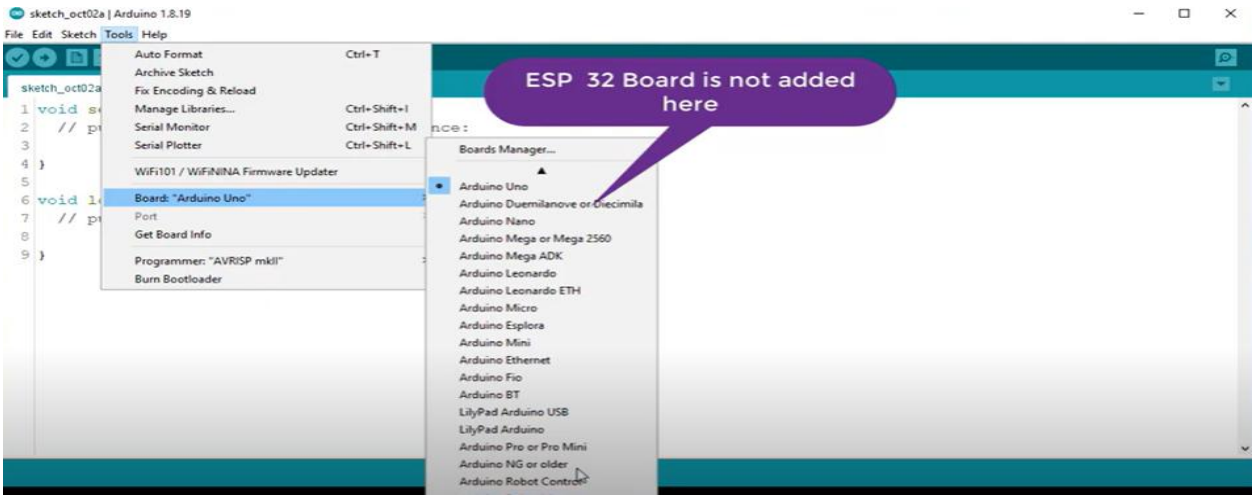
STEP 2: Install Arduino IDE:

Windows: Run the downloaded executable file. Follow the on-screen prompts, agreeing to the license terms and selecting installation options (e.g., install USB driver).

macOS: Drag the downloaded Arduino application file into your Applications folder.

Linux: Extract the downloaded archive and run the arduino-ide executable within the extracted folder.

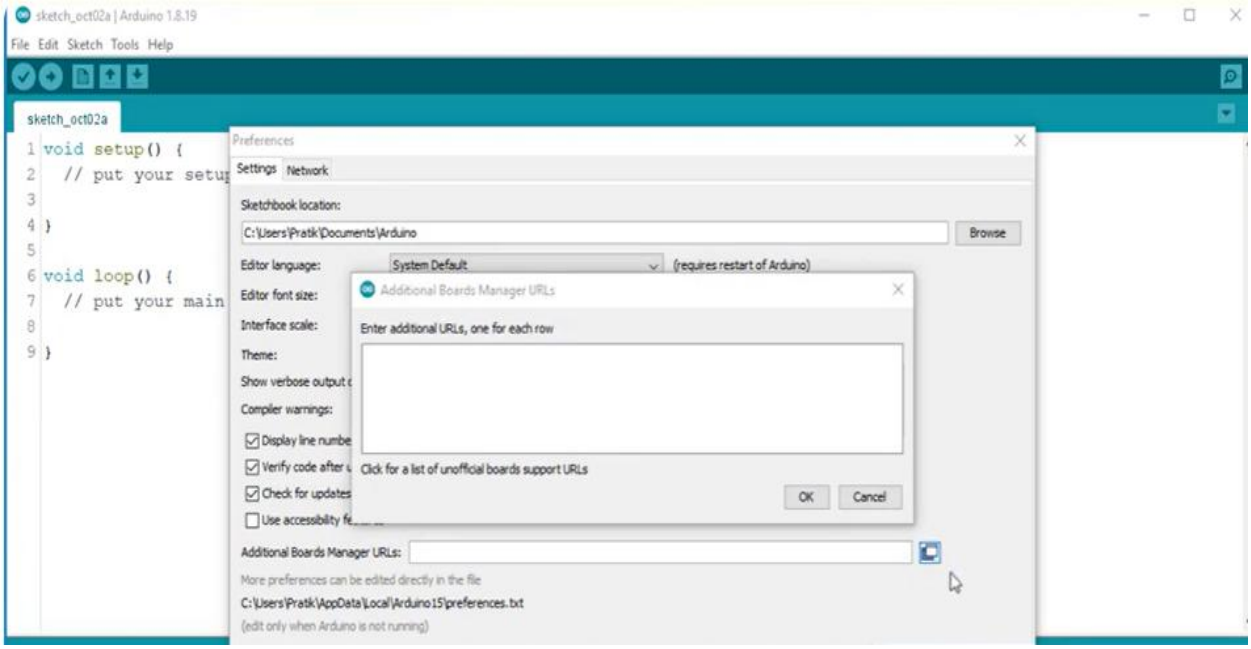
STEP 3: Launch Arduino IDE: Open the Arduino IDE application after the installation is complete.



Installing ESP32 Board Support in Arduino IDE:

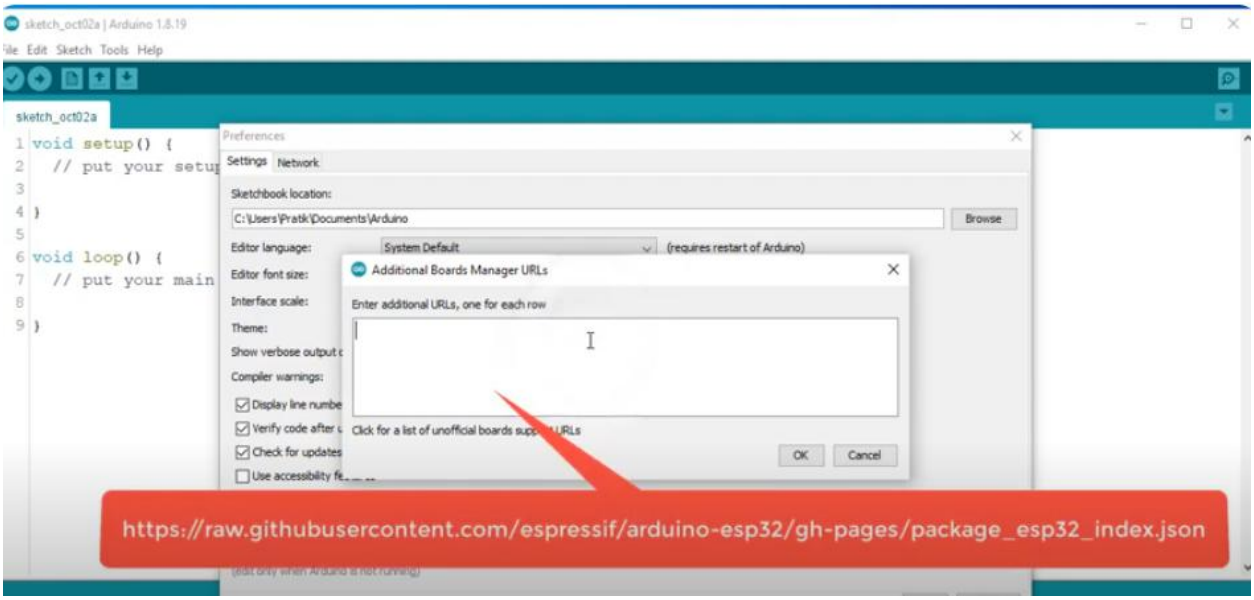
- **Add ESP32 Board Manager URL:**

STEP 1: In the Arduino IDE, go to File > Preferences.



STEP 2: In the "Additional Boards Manager URLs" field, paste the following URL:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

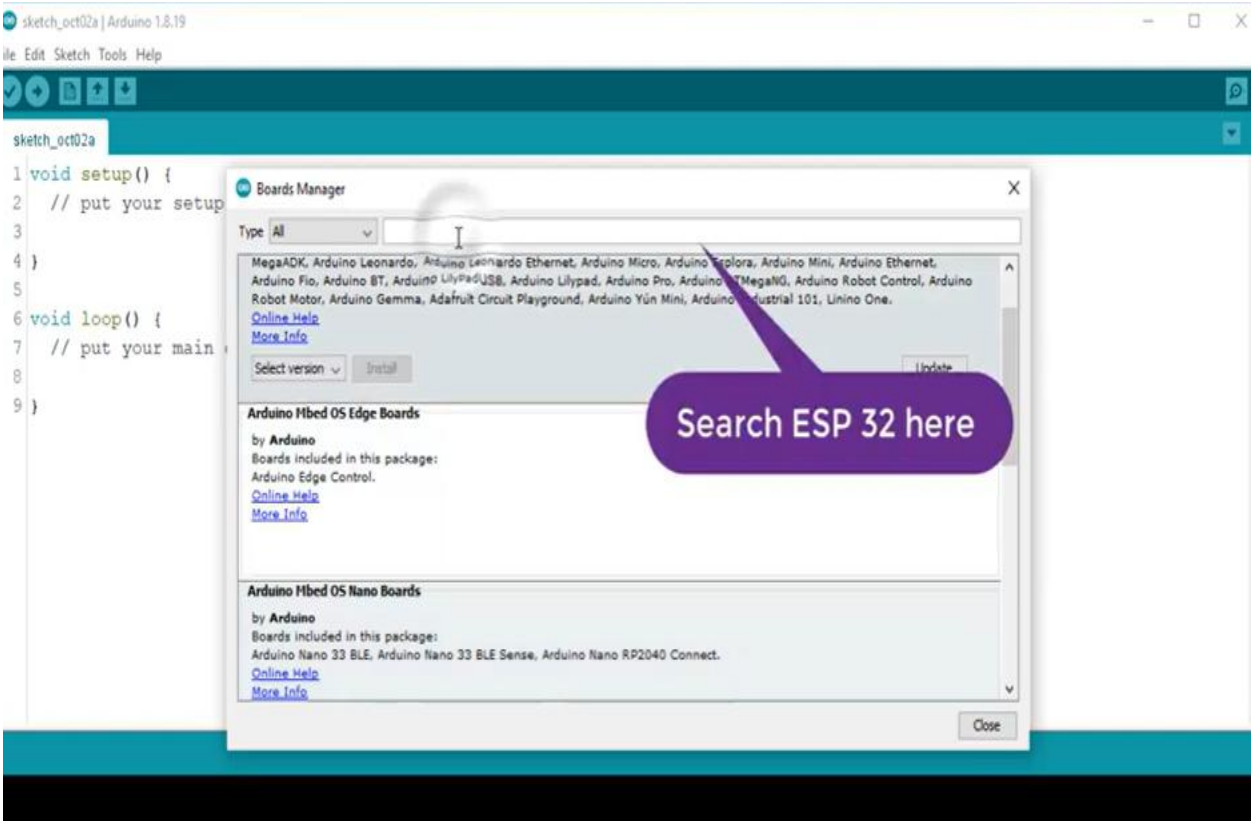


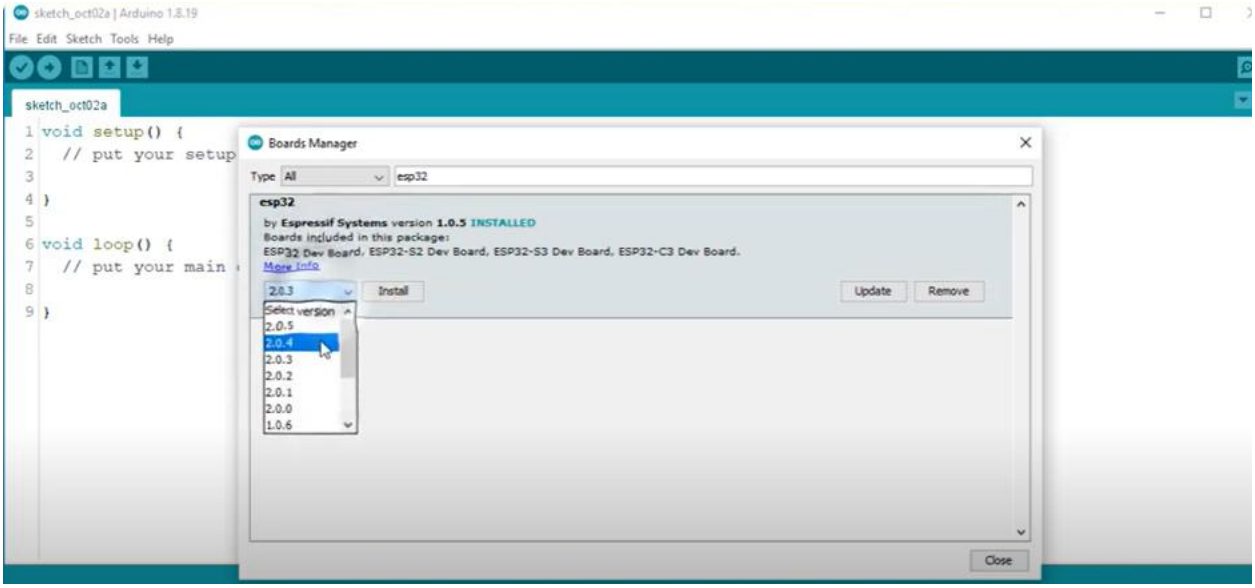
STEP 3: Click "OK".

Note: If you already have other URLs (e.g., ESP8266), separate them with a comma.

- **Open Boards Manager:**

Go to Tools > Board > Boards Manager….



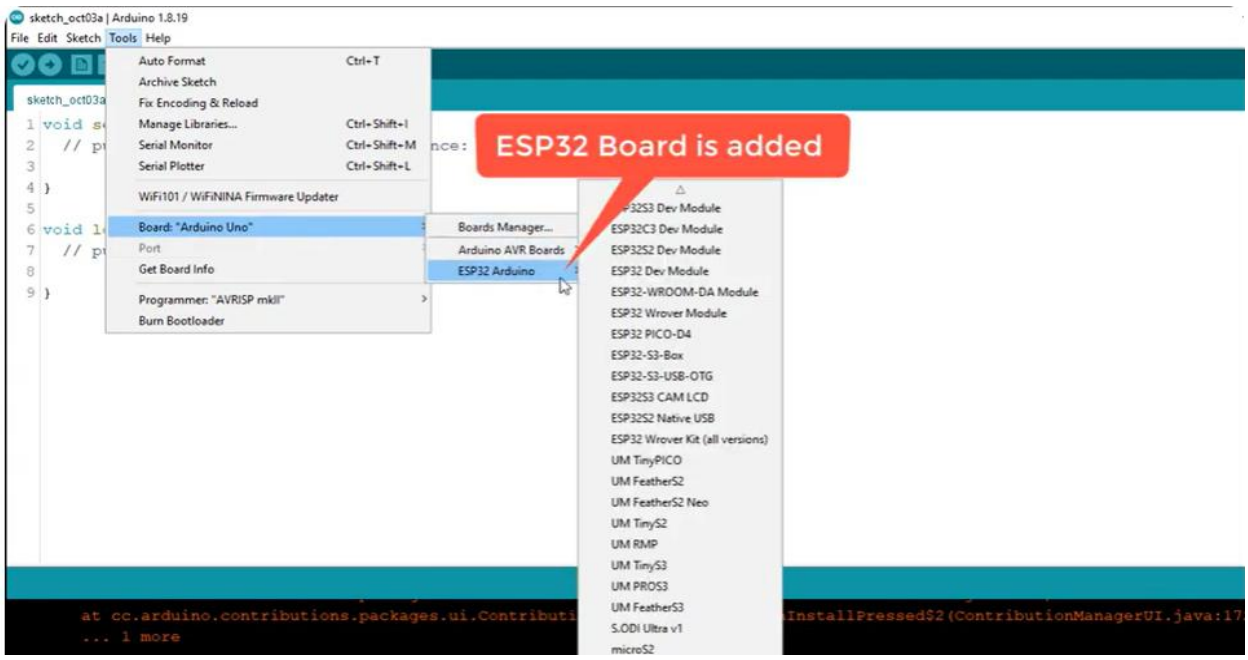


Install ESP32 Boards:

- In the Boards Manager search bar, type "ESP32".
- Locate "ESP32 by Espressif Systems" and click the "Install" button.

Select ESP32 Board:

After the installation completes, go to Tools > Board and you should now see a list of ESP32 boards. Select the specific ESP32 board you are using (e.g., "ESP32 Dev Module", "ESP32 WROOM-32").



- **Select Serial Port:**

  Connect your ESP32 board to your computer via a USB cable. Go to Tools > Port and select the COM port corresponding to your connected ESP32 board. (You may need to check your system's Device Manager or equivalent to identify the correct port).

  Uploading a Sketch to ESP32:

  - **Open or Write a Sketch:**

  Open an existing ESP32 example sketch (e.g., File > Examples > ESP32 > Blink) or write your own code.

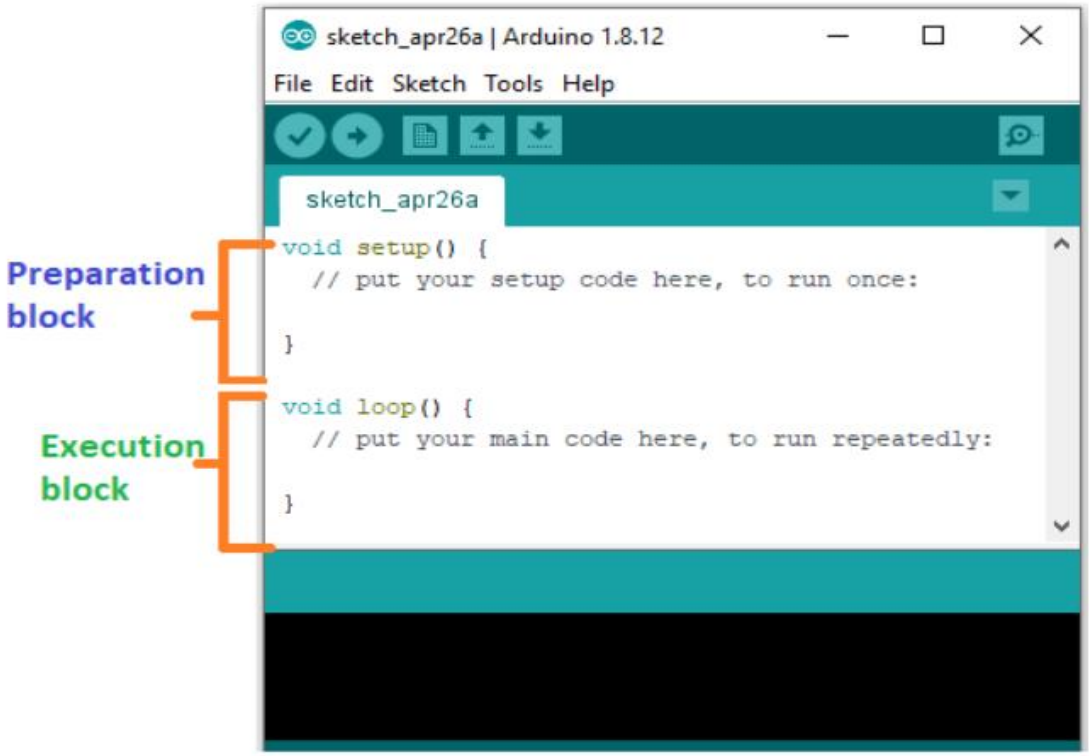
  - **Upload:**

  Click the "Upload" button (right arrow icon) in the Arduino IDE. You might need to press and hold the "BOOT" button on your ESP32 board during the upload process, as indicated by the IDE's status messages. Release the "BOOT" button once the upload starts.

- **Monitor Output (Optional):**

  Open the Serial Monitor (Tools > Serial Monitor) to view any serial output from your ESP32.

BASIC PROGRAMMING CONCEPTS OF ARDUINO IDE:



1. pinMode()

- Purpose: Configures a specific pin to behave either as an input or output.
- Syntax:
  pinMode(pin, MODE);
  MODE can be INPUT, OUTPUT, INPUT_PULLUP
- Example:
  pinMode(13,OUTPUT);
  Usage: Always set the pin mode in the setup() function before reading from or writing to a pin.

2. digitalWrite() and digitalRead()

- digitalWrite(): Sets a digital pin to HIGH or LOW.
  digitalWrite(pin, HIGH);
  digitalWrite(pin, LOW);
- digitalRead(): Reads the state of a digital pin (HIGH or LOW).
  int buttonState = digitalRead(7);
- Usage:
  Use digitalWrite for controlling LEDs, relays, or other devices.
  Use digitalRead for reading switches or sensors that provide binary signals.

3. analogRead()

- Purpose: Reads the analog voltage (0 to 5V or 0 to 3.3V depending on board) and converts it to a number between 0 and 1023.
- Syntax:
  int sensorValue = analogRead(A0);
- Usage: Typical for sensors like potentiometers or temperature sensors.

4. analogWrite()

- Purpose: Writes a PWM (Pulse Width Modulation) signal to a pin, simulating an analoge output.
- Syntax:
  analogWrite(pin, Value);
- Usage: Controlling LED brightness or motor speed.

5. delay()

- Purpose: Pauses program execution for a specified number of milliseconds.
- Syntax:
  Delay(1000);
- Usage: Simple timing between actions.

RESULTS:

CONCLUSION:

Assignment Question

1) What are the main features of the ESP32, and how does it differ from other microcontrollers like the Arduino Uno or NodeMCU?

2) How do you set up and upload a simple program, like blinking an LED, to the ESP32 using the Arduino IDE?

3) How can you connect the ESP32 to a Wi-Fi network?

4) What is the role of the GPIO pins on the ESP32, and how can you use them to control sensors or actuators in a project?